Benchmarking AES-128 on a MicroBlaze Soft-Core Processor

Nexys A7-100T FPGA Platform

Samuel Regan · David Jessup · Nico Johnson

Kenneth Wang · Edwin Estrada

Project idea

- Benchmark **AES-128** on a **MicroBlaze** soft-core processor
- Run the design on a **Nexys A7-100T FPGA**
- Measure how architecture affects cryptographic performance

Why AES?

- AES is a major real-world **cryptographic standard**
- Used in secure communication, storage, Wi-Fi, banking, and authentication
- Practical workload for studying processor efficiency

Why this project?

- Connects software behavior to hardware execution
- Lets us analyze **cycles, latency, and throughput**
- Shows trade-offs of a soft-core processor vs dedicated hardware

Steps used to complete the project

- Build a working **MicroBlaze + UART** platform in Vivado/Vitis
- Verify the software flow with a simple output test
- Integrate a known-good **AES-128** implementation
- Validate correctness using **AES test vectors**
- Add a timer-based benchmark harness
- Run tests for multiple input sizes
- Record and graph the measured results

Tested cases

- Encryption
- Decryption
- Combined encryption + decryption

Total cycles

$$\text{Total Cycles} = \text{End Counter} - \text{Start Counter}$$

Measures total processor work required.

Cycles per byte

$$\text{Cycles/Byte} = \frac{\text{Total Cycles}}{\text{Input Size (Bytes)}}$$

Measures efficiency per byte processed.

Latency

$$\text{Latency} = \frac{\text{Total Time}}{\text{Number of Iterations}}$$

Measures time to complete one operation.

Throughput

$$\text{Throughput} = \frac{\text{Input Size (Bytes)}}{\text{Latency}}$$

Measures data processed per unit time.

- **Tool consistency:** all team members had to use the same **Vivado/Vitis version**; mismatched versions caused build incompatibilities
- **Timing accuracy:** UART print overhead had to be kept **outside** the timed region
- **Correctness first:** AES output was verified using **standard test vectors** before benchmarking
- **Stable timing:** iteration counts had to be chosen carefully to reduce noise without making runs impractical

Key lesson

- Reliable benchmarking depends on both **correct code** and a **controlled measurement setup**

Input sizes tested

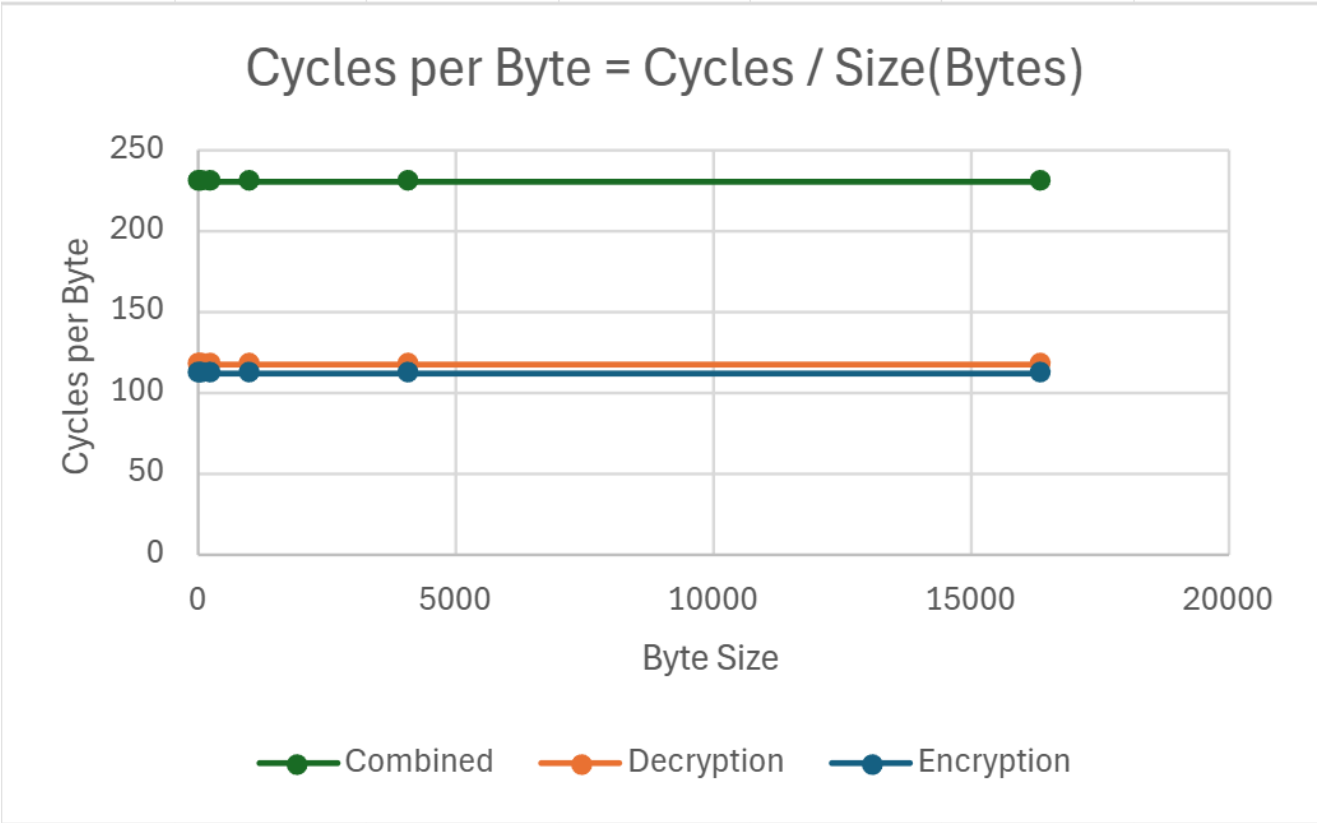
- 16, 64, 256, 1024, 4096, and 16384 bytes

Main measured trends

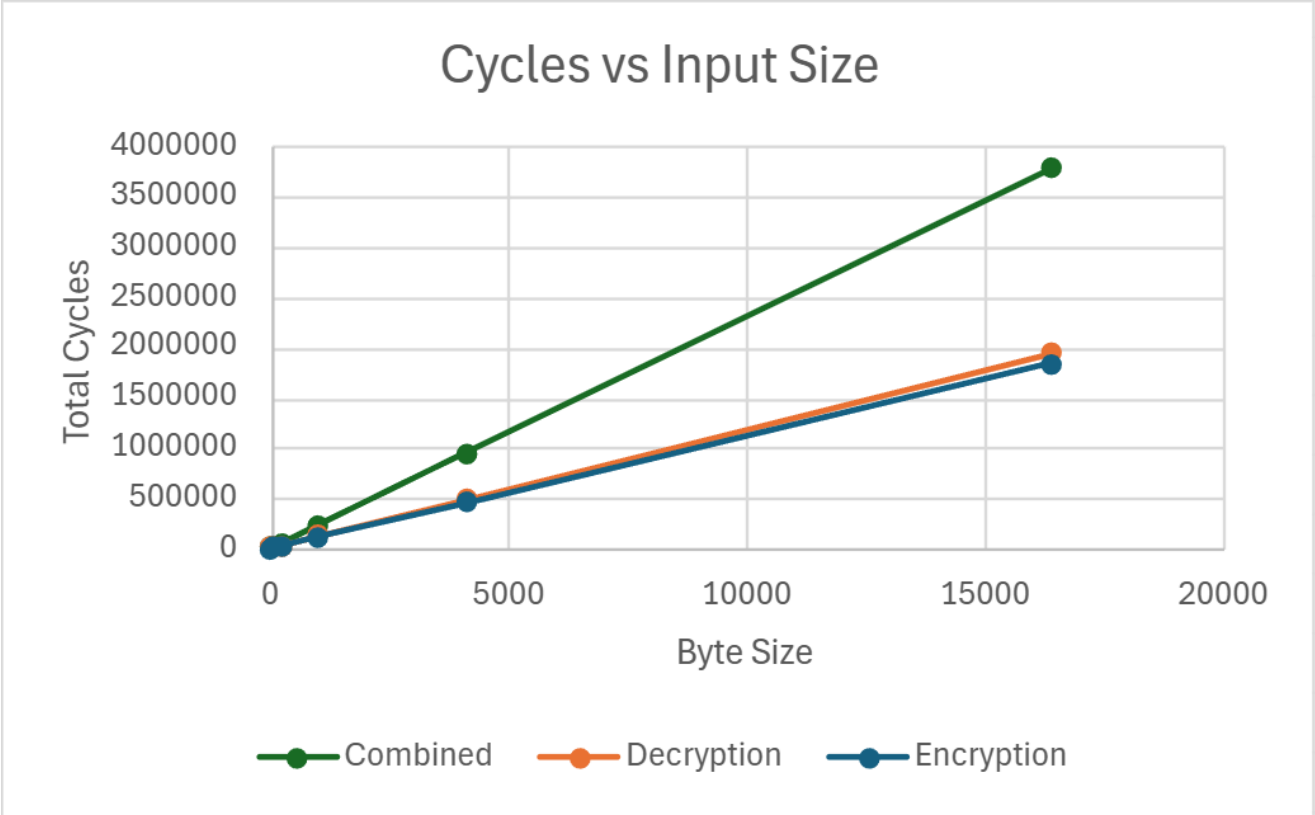
- **Cycles per byte** stayed nearly constant
- **Total cycles** increased linearly with input size
- **Throughput** stayed nearly constant across all sizes
- **Latency** increased linearly with input size

Performance ordering

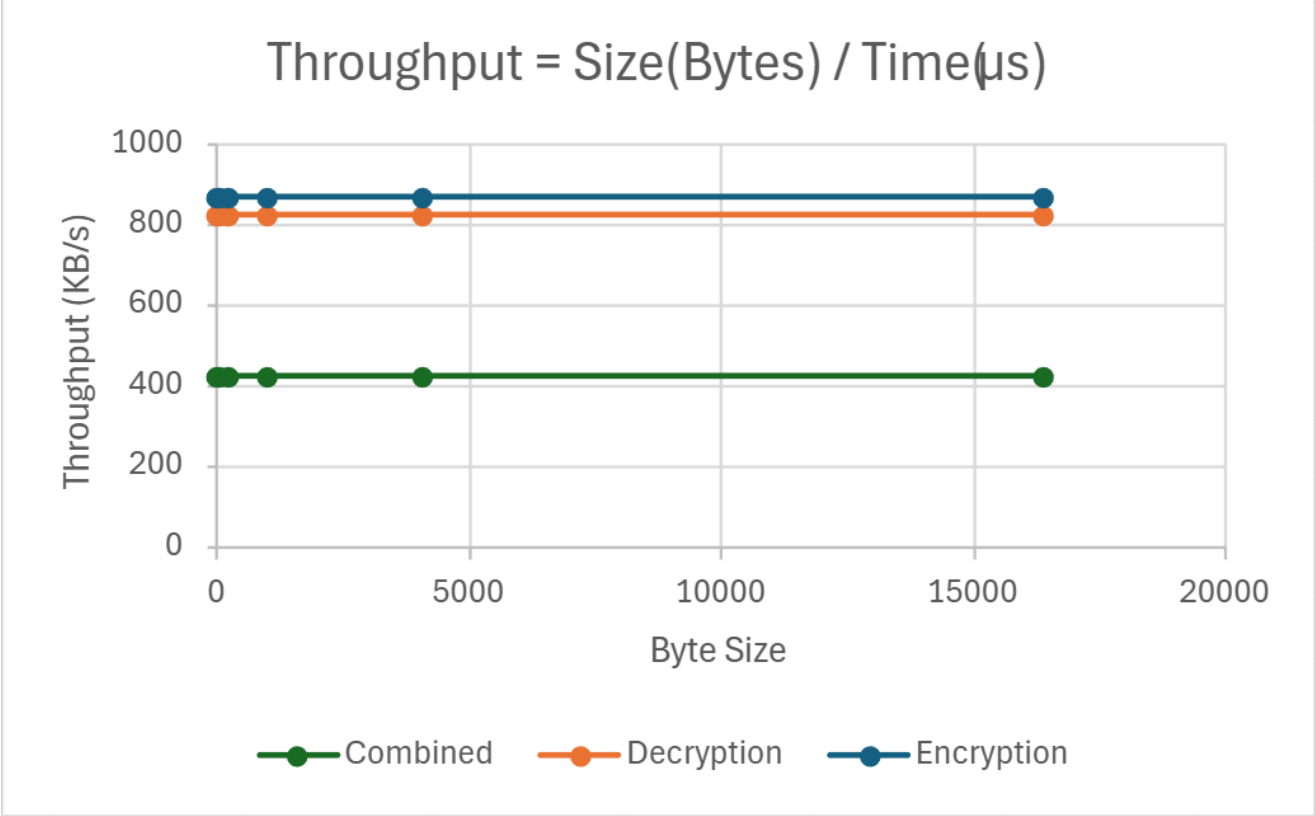
- **Encryption** performed best
- **Decryption** was slightly slower
- **Combined** cost was highest, as expected



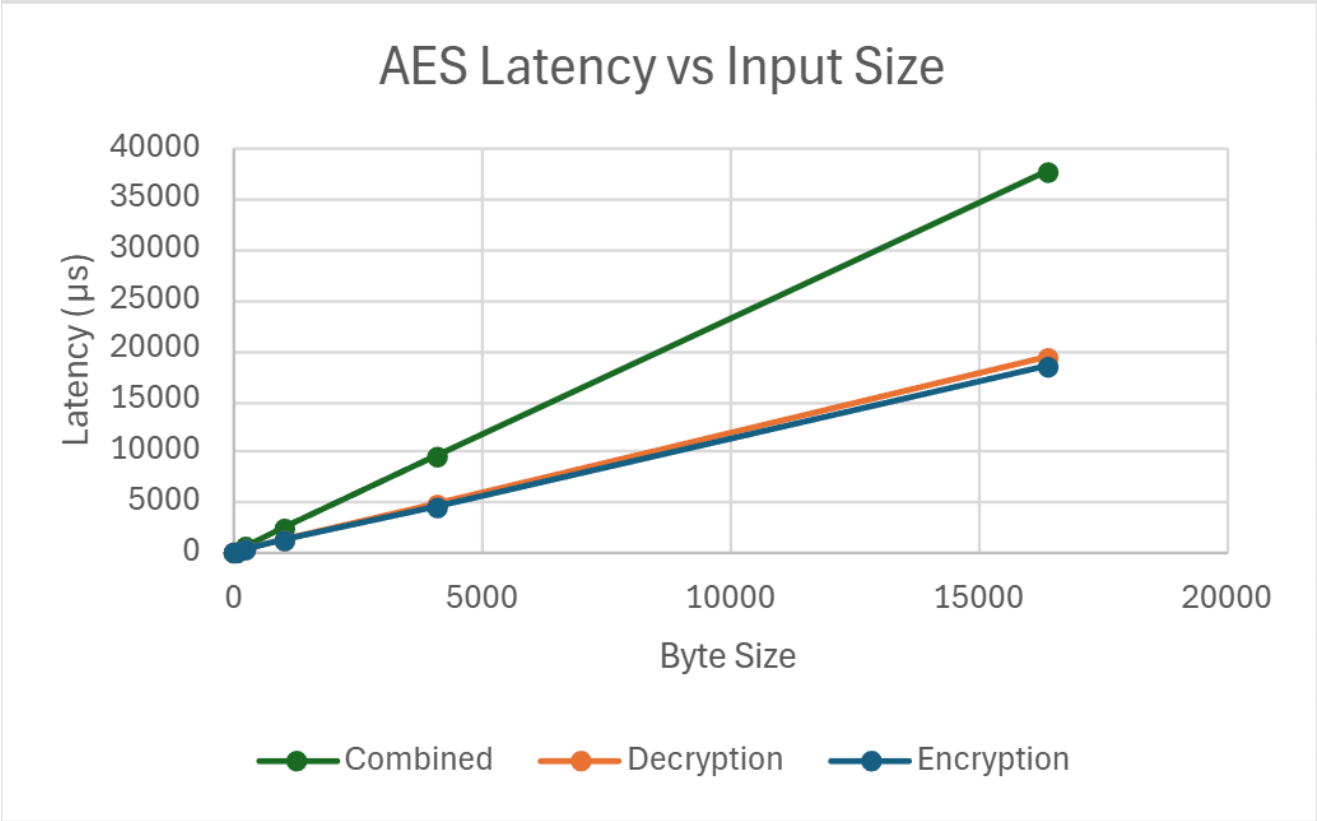
- Flat lines show **stable efficiency** across all input sizes
- Encryption is lowest (**112 cycles/byte**); decryption is slightly higher (**118**)
- Combined is highest because it includes both operations



- Total cycles increase **linearly** as input size grows
- AES work scales proportionally with data size
- Encryption has the smallest slope; combined has the largest



- Throughput stays nearly **constant** across all tested sizes
- Encryption is highest at about **868 KB/s**
- Combined is lowest because encryption and decryption are both included



- Latency increases **linearly** with input size
- Larger messages require more AES block operations
- Encryption stays faster than decryption; combined is highest

What we showed

- AES-128 was successfully benchmarked on a **MicroBlaze soft-core** on the **Nexys A7-100T**
- The implementation showed **predictable, scalable performance**
- Cycles per byte stayed constant, while cycles and latency grew linearly
- Throughput remained stable across the tested data sizes

Architecture takeaway

- The project connected **software execution** to **hardware-level performance metrics**
- It highlighted the trade-off between **FPGA flexibility** and **soft-core performance cost**

Final result

- Encryption was the most efficient case, decryption was slightly slower, and combined had the highest cost as expected